

**Universidad Nacional de Ingeniería
Facultad de Ciencias**

Arquitectura de computadores

El Procesador

Prof.: Lic. César Martín Cruz S.
ccruz@uni.edu.pe

2015 – I

Introducción

- En esta parte del curso se tratará de :
 - El **ALU** y los elementos que compone.
 - Las principales técnicas usadas en el diseño de un procesador.
 - La construcción del **datapath** y del **control**.
 - Estudiaremos la implementación de una versión reducida de **MIPS**.

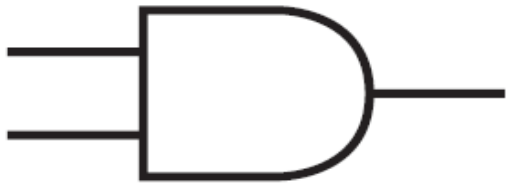
Definición

- La **ALU** (unidad aritmético-lógica) es el dispositivo que se encarga de realizar:
 - a) Operaciones aritméticas (suma, resta, etc.).
 - b) Operaciones lógicas (not, and, or, xor, etc.).
 - c) Operaciones de desplazamiento.

Circuitos combinatorios

- Su salida depende exclusivamente de sus entradas.

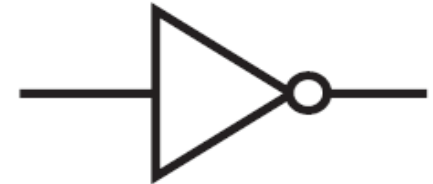
Compuertas básicas



AND



OR



NOT

AND		
A	B	$A \cdot B$
0	0	0
0	1	0
1	0	0
1	1	1

OR		
A	B	$A + B$
0	0	0
0	1	1
1	0	1
1	1	1

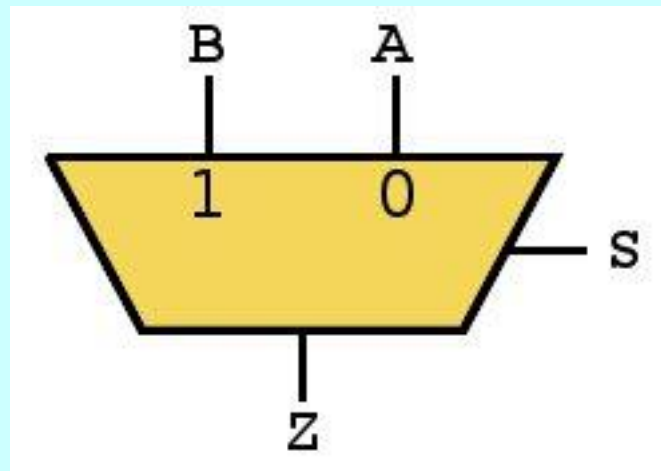
NOT	
A	$\bar{A}$
0	1
1	0

Otras compuertas

- **XOR** (or exclusivo).
- **EQV** (equivalence).
- **NAND** (not AND).
- **NOR** (not OR).

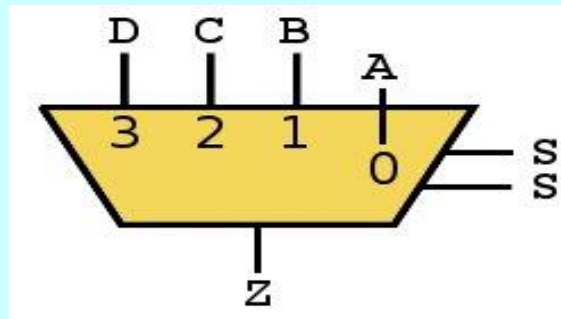
Multiplexor

- El multiplexor (mux) tiene 2^n entradas de datos, n bits de selección y una salida.
- Los bits de selección se usan para decidir cuál entrada pasa a la salida.
- Mux 2 a 1

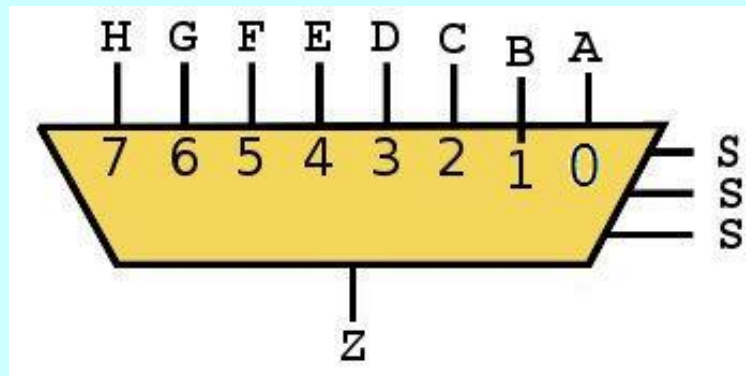


Multiplexor

- Mux 4 a 1

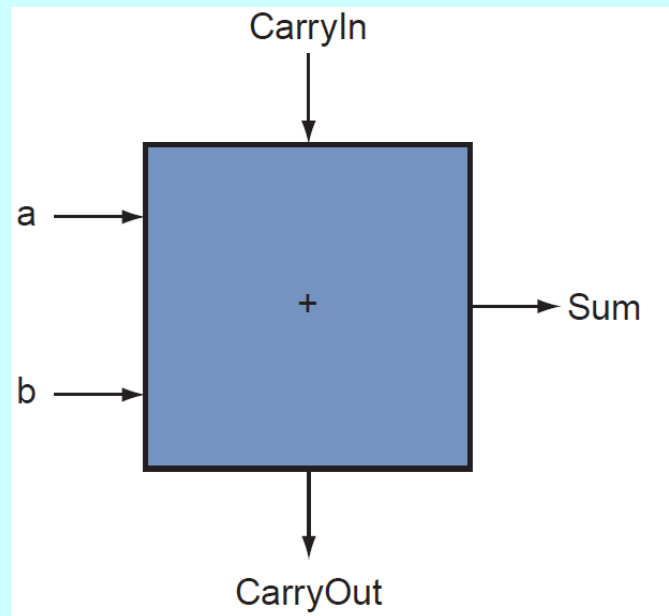


- Mux 8 a 1



Sumador completo

- Sumador completo (full adder) de 1 bit:
 - Entradas: dos números de 1 bit y un bit de carry de entrada.
 - Salidas: la suma de 1 bit y un bit de carry de salida.

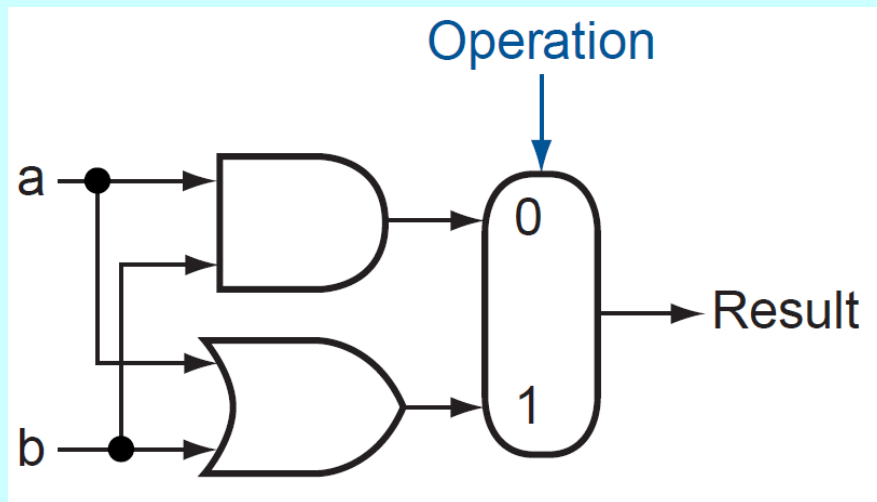


Sumador completo

Inputs			Outputs	
a	b	CarryIn	CarryOut	Sum
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Un ALU de 1 bit

- Dos operaciones: AND y OR.
- Un bit para seleccionar la operación entre AND y OR.



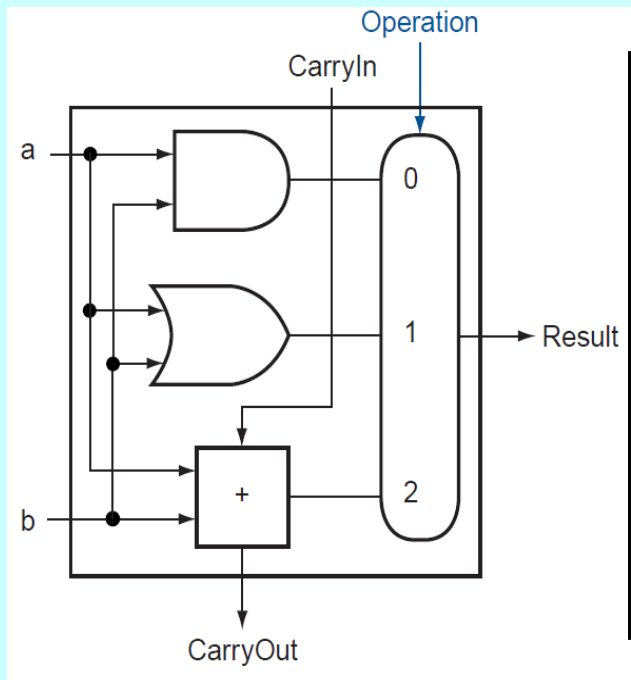
Operación	Resultado
0	$a \cdot b$
1	$a + b$

Agregando operaciones

- El siguiente paso es agregar la suma.
- Se agrega un sumador completo al diseño anterior.

ALU de 1 bit

- 3 operaciones: AND, OR y suma.
- 2 bits para seleccionar la operación.



Operación	Resultado	CarryOut
00	$a \cdot b$	X
01	$a + b$	X
10	$a \oplus b \oplus \text{CarryIn}$	$(a + b) \cdot \text{CarryIn} + (a \cdot b)$
11	X	X

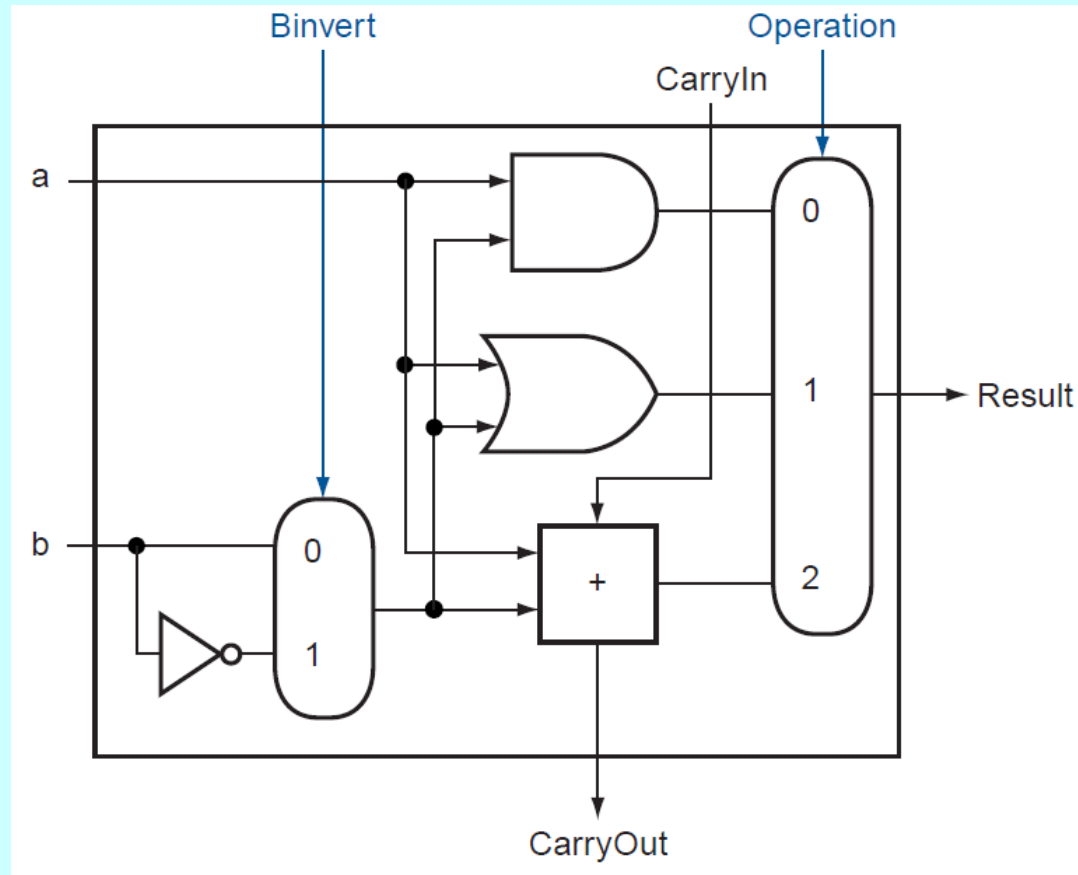
Agregando operaciones

- Agregar la resta $a - b$.
- $a - b \equiv a + (\text{el complemento a 2 de } b)$.
- El complemento a dos de b se encuentra sumando 1 al complemento a uno de b .
- El complemento a uno de b se encuentra negando a b .
- $a - b = a + (\neg b + 1) = a + \neg b + 1$
- El 1 de la suma viene en CarryIn.

ALU de 1 bit

- 4 operaciones: AND, OR, suma y resta.
- 2 bits para seleccionar la operación.
- Un bit extra para diferenciar entre la suma y la resta.
- En la resta, CarryIn es 1.

ALU de 1 bit



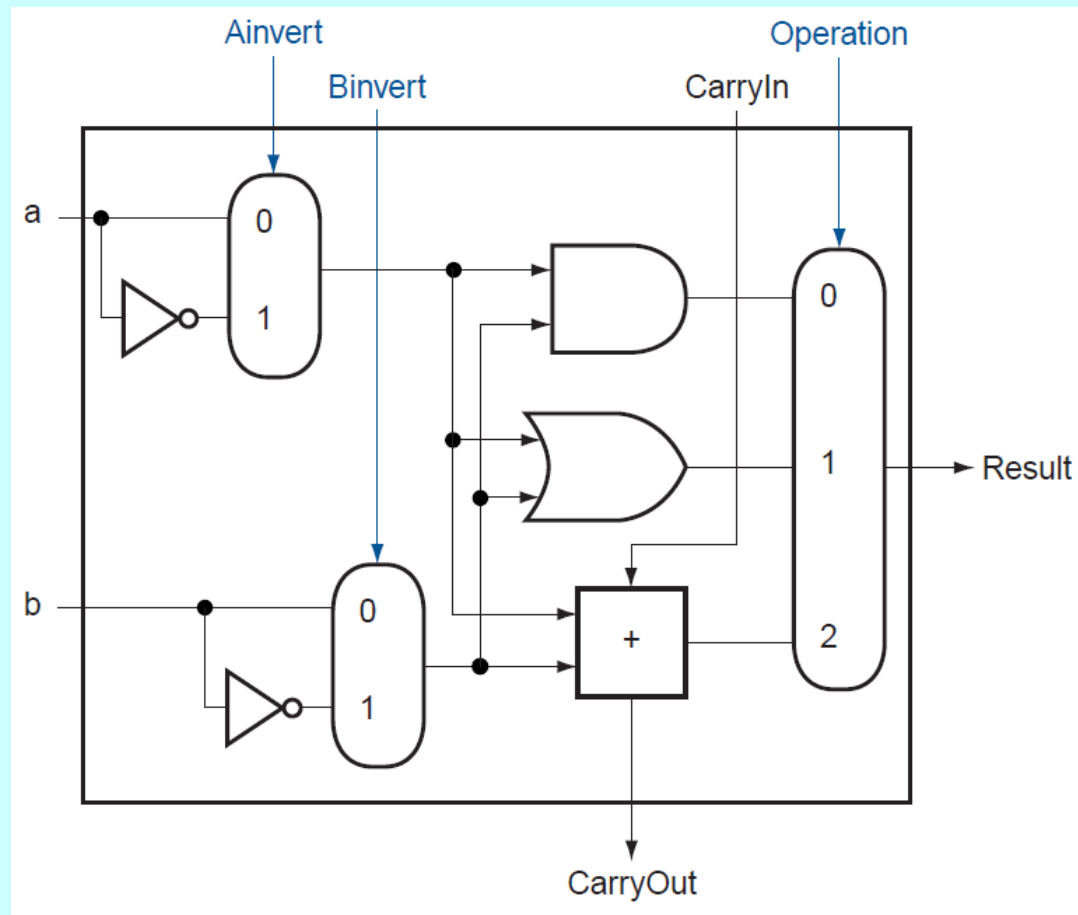
Agregando operaciones

- Agregar la operación NOR:
 - $\neg(a + b)$
- Ley de DeMorgan:
 - $\neg(a + b) = \neg a \cdot \neg b$
- La ALU ya puede calcular $a \cdot b$ y $\neg b$.
- Hace falta poder calcular $\neg a$.

ALU de 1 bit

- 5 operaciones: AND, OR, NOR, suma y resta.
- 2 bits para seleccionar la operación.
- Binvert diferencia entre la suma y la resta.
- En la resta, CarryIn es 1.
- Ainvert y Binvert diferencian entre AND y NOR.

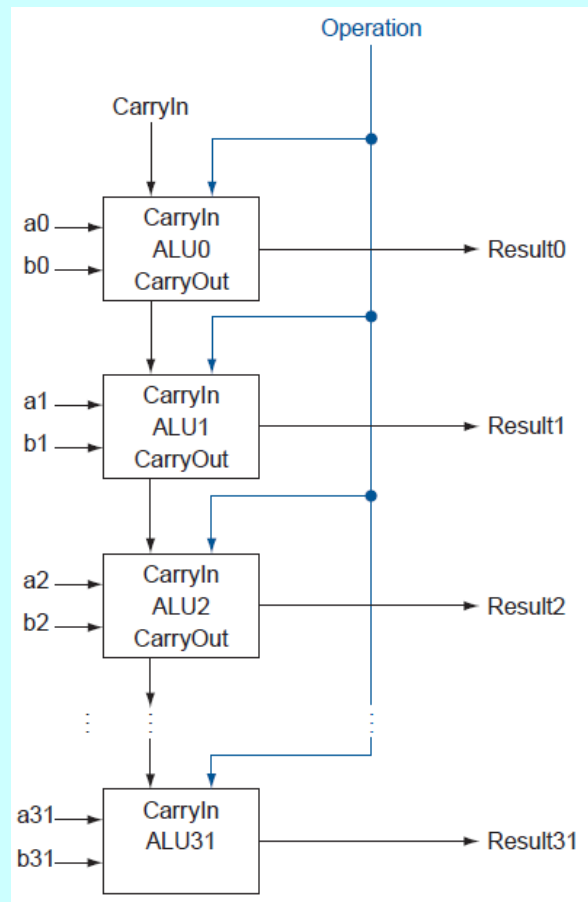
ALU de 1 bit



ALU de 32 bits

- ¿Cómo se genera una ALU de 32 bits?
 - Con 32 ALUs de 1 bit.
 - CarryOut de la ALU_i se conecta a CarryIn de la ALU_{i+1} .
 - En la resta CarryIn ALU_0 se conecta a 1.

ALU de 32 bits

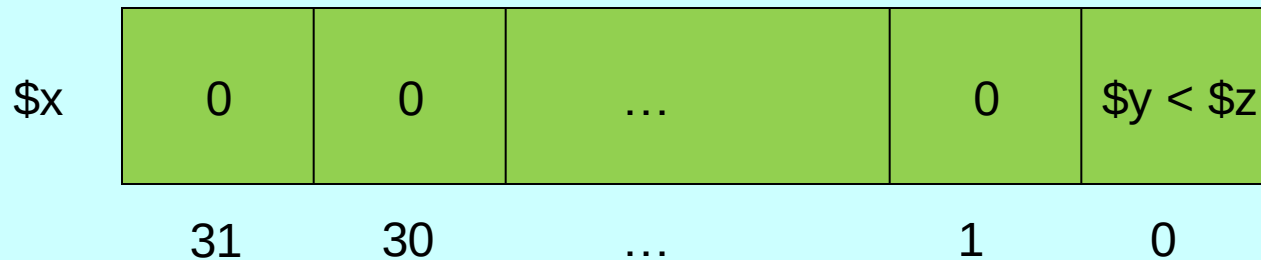


ALU para MIPS

- El diseño de la ALU anterior está incompleto.
- Se necesita soportar la instrucción **slt** (set on less than).
- `slt $x, $y, $z` guarda 1 en \$x si $y < z$ y 0 en otro caso.

Soportando slt

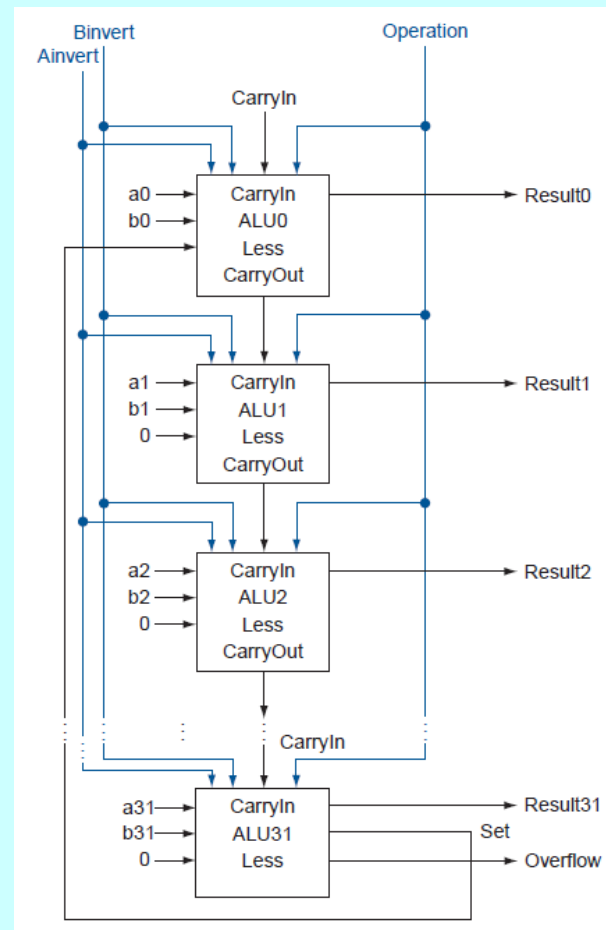
- slt \$x, \$y, \$z pone a ceros los bits desde 1 al 31 de \$x.
- El bit 0 de \$x tiene el resultado de la comparación de \$y y \$z.



Soportando slt

- Se calcula $t = \$y - \z .
- Si t es negativo $\$y < \z .
- Si t es positivo o cero $\$y \geq \z .
- En MIPS los números negativos tienen 1 en el bit 31.
- El bit 31 de t tiene el resultado de la comparación.

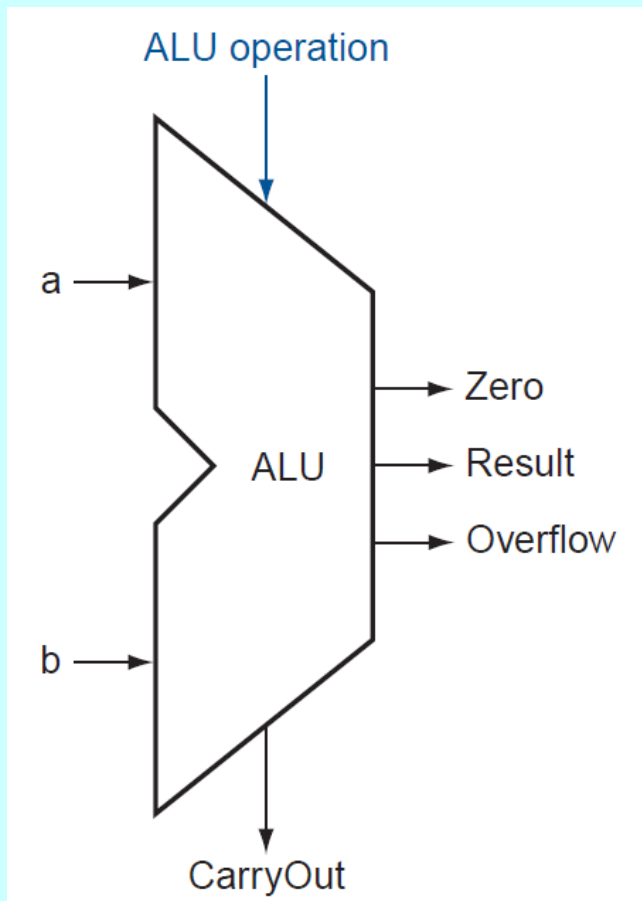
ALU de 32 bits



ALU para MIPS

- Faltan soportar los **saltos condicionales**.
 - beq \$x, \$y, L – salta a L si $\$x = \y .
 - bne \$x, \$y, L – salta a L si $\$x \neq \y .
 - Se calcula $t = \$x - \y .
 - Si t es cero, $\$x = \y .
 - Si t no es cero, $\$x \neq \y .

Diagrama y tabla de la ALU



Líneas de control				Función
C ₃	C ₂	C ₁	C ₀	
0	0	0	0	AND
0	0	0	1	OR
0	0	1	0	suma
0	1	1	0	resta
0	1	1	1	set on less than
1	1	0	0	NOR
C ₃ = Ainvert				
C ₂ = Bnegate				

El procesador

- El procesador o CPU (unidad central de procesamiento) sigue las instrucciones del programa al pie de la letra. Suma y compara números, realiza operaciones lógicas, ordena activarse y desactivarse a los dispositivos de I/O, etc.
- El procesador consta de dos componentes:
 - El **datapath**. Ejecuta operaciones aritméticas y lógicas.
 - El **control**. Ordena al datapath, memoria y dispositivos de Entrada/Salida lo que hay que hacer de acuerdo al programa.

Construyendo un Datapath

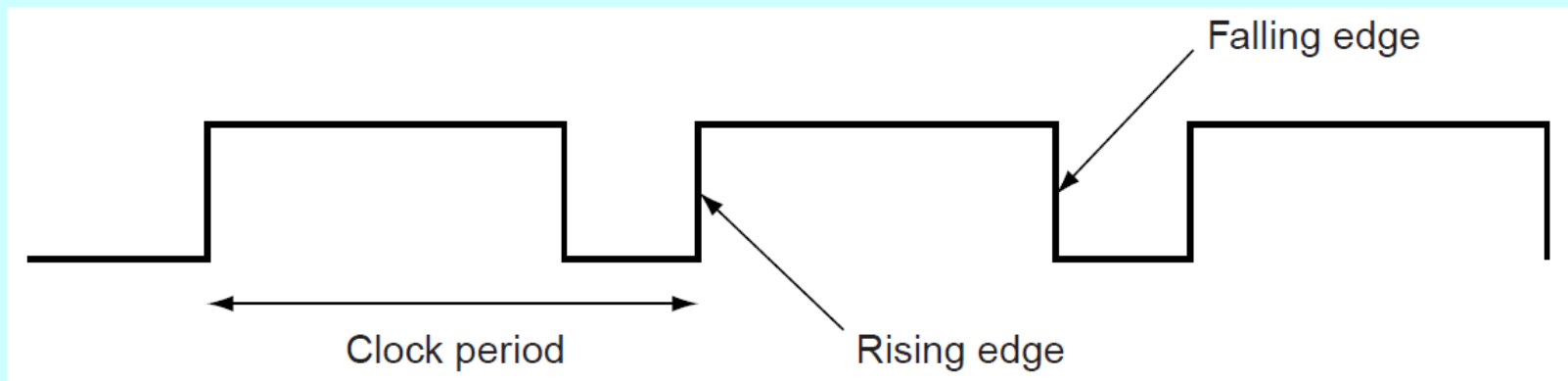
- **Datapath.** Consiste de elementos que procesan datos y direcciones dentro del CPU. Estos son:
 - ❖ Registros, ALUs, multiplexores, memorias,
- Construiremos un datapath de MIPS incrementalmente.

Repasando circuitos digitales

- Hay dos clases de circuitos digitales:
 1. **Circuitos combinatorios.** La salida depende solo de las entradas. Ejemplos : AND, OR, NOT, decoders, multiplexores, sumadores, etc.
 2. **Circuitos secuenciales.** La salida depende de las entradas y de la salida actual. Ejemplos : Latches y flip-flops.

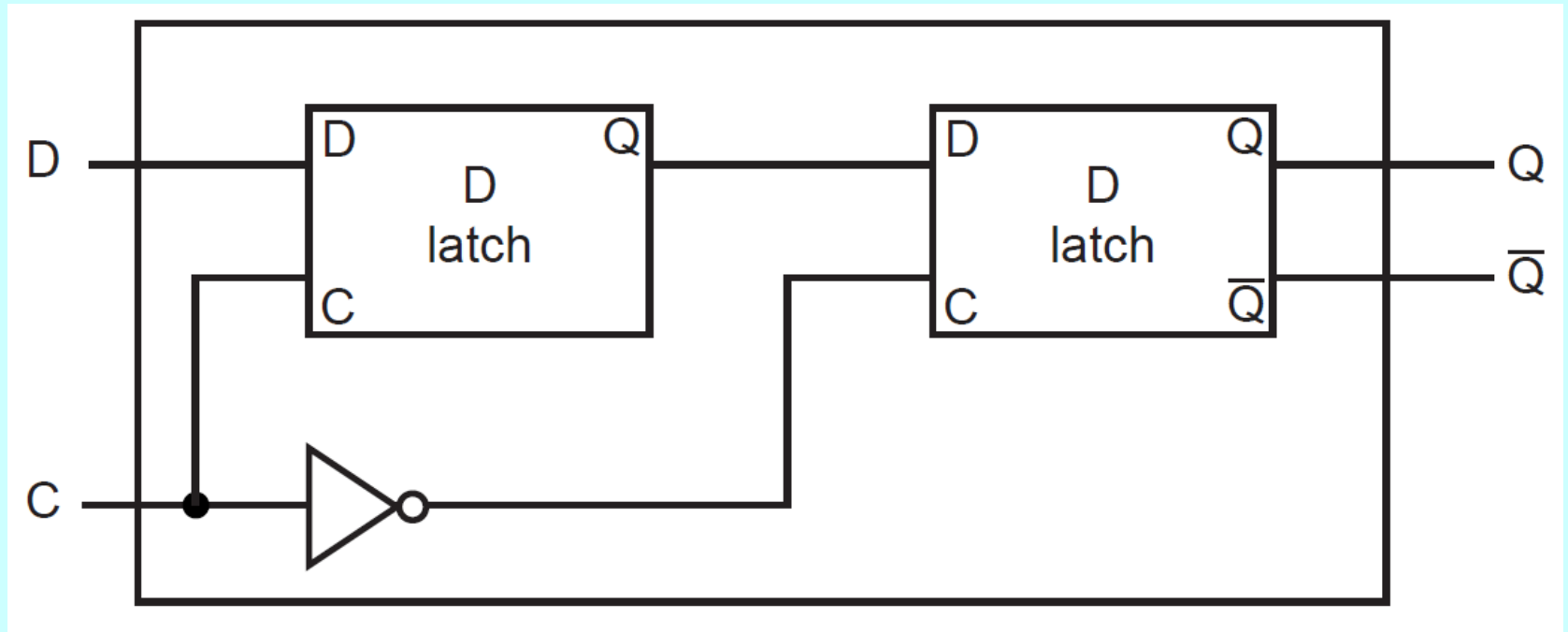
Circuitos secuenciales

- Pueden almacenar 1 bit.
- Usaremos solo flip-flops (biestables) maestro-esclavo. La salida se actualiza durante el flanco (edge) de reloj.



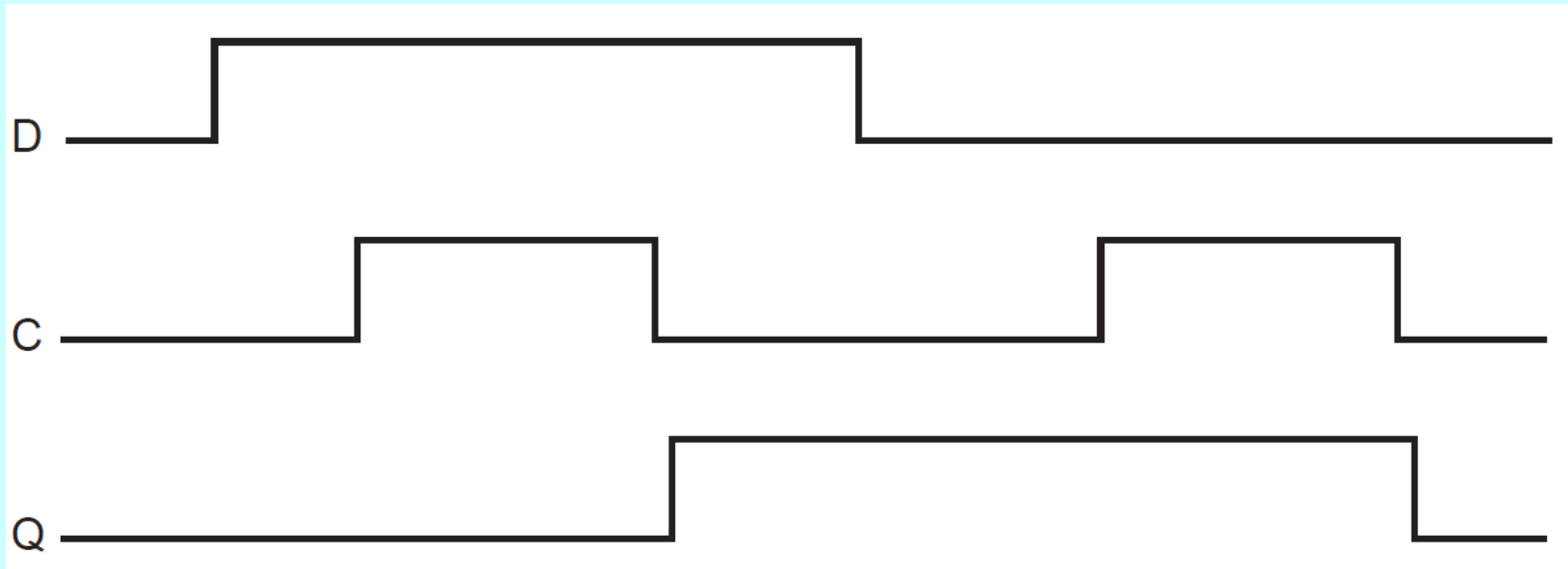
Flip-flop D

- Diagrama de un **flip-flop D** maestro-esclavo disparado por el flanco de bajada:



Flip-flop D

- Operación:

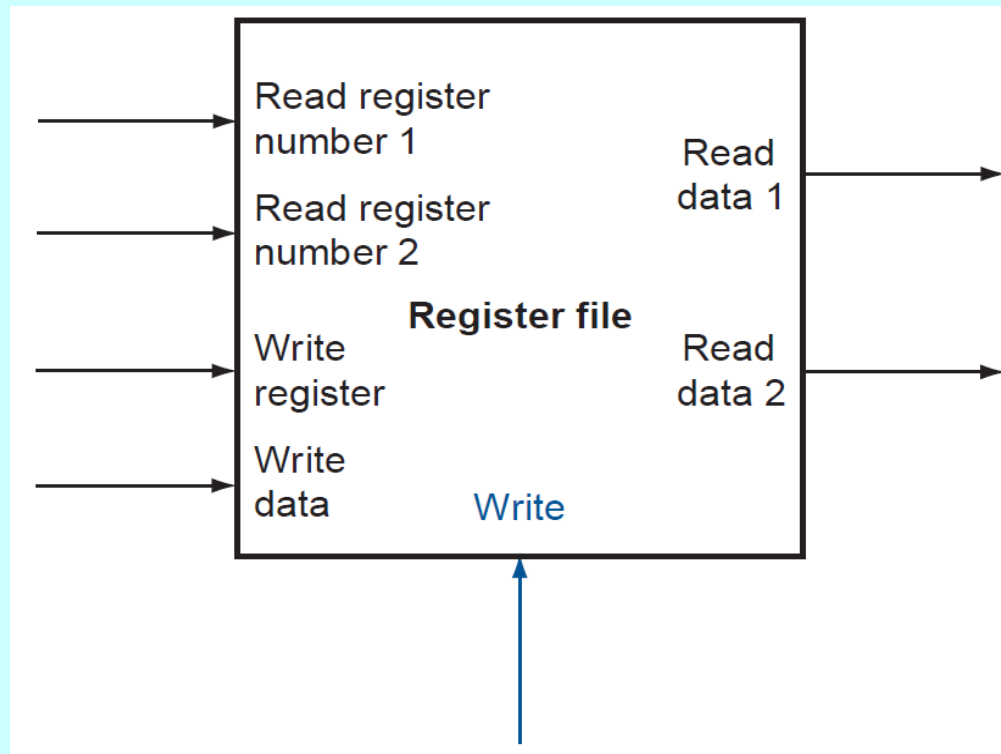


Banco de registros

- El banco de registros (*register file*) es un conjunto de registros para guardar y leer datos.
- Cada registro es un **vector de flip-flops D**.
- Para leer un registro:
 - Entrada: número de registro.
 - Salida: dato contenido en el registro.
- Para escribir un registro:
 - Entrada: número de registro, dato y una señal de reloj para controlar la escritura.

Banco de registros

- Dos puertos de lectura y uno de escritura.



MIPS simplificado

- Las instrucciones se hacen en un ciclo de reloj.
- Comienzan a ejecutarse en un flanco de reloj y terminan en el siguiente flanco.

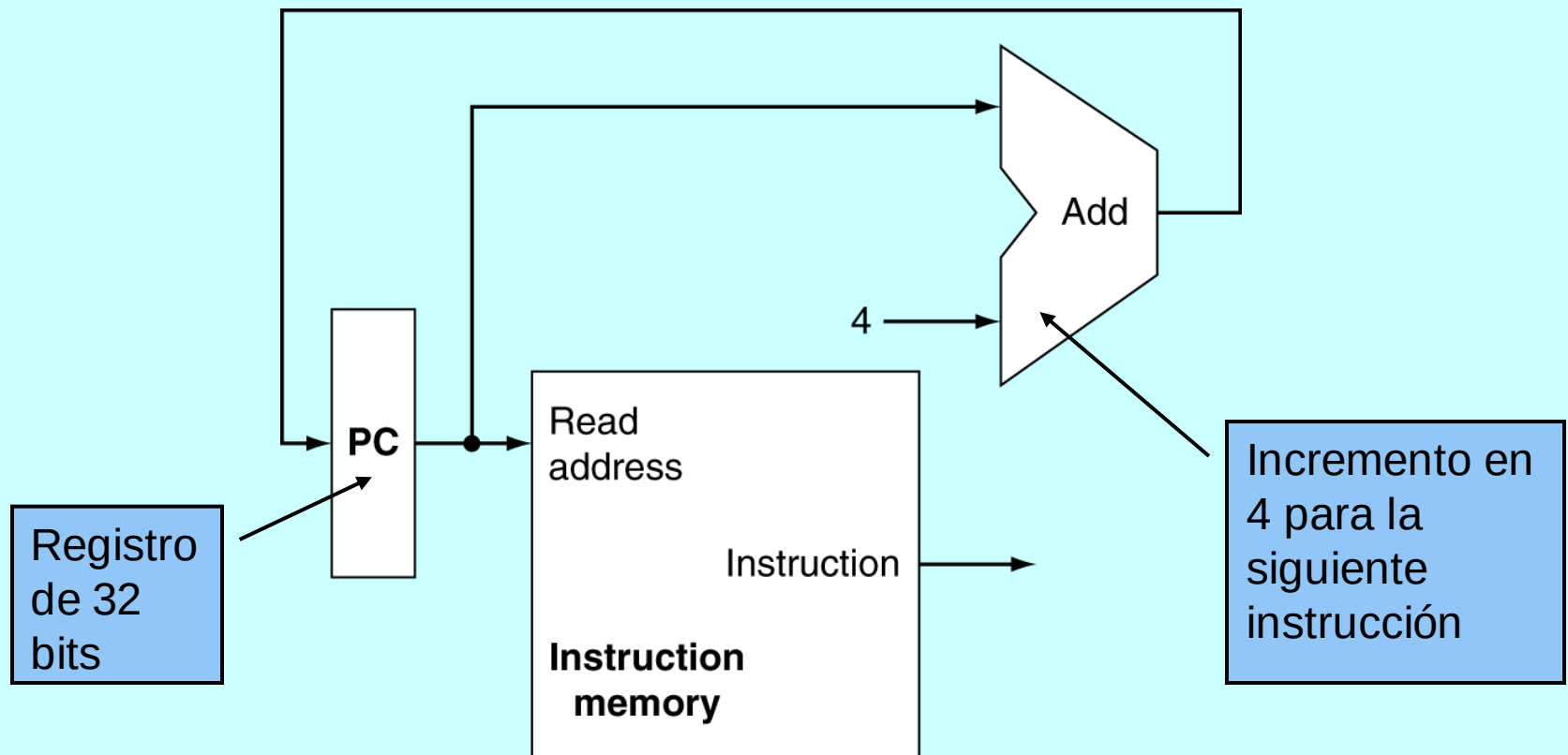
MIPS simplificado

- Tres tipos de instrucciones:
 1. Instrucciones de referencia a memoria: load word (lw) y store word (sw).
 2. Instrucciones aritmético-lógicas: suma (add), resta (sub), and, or y set on less than (slt).

Implementación

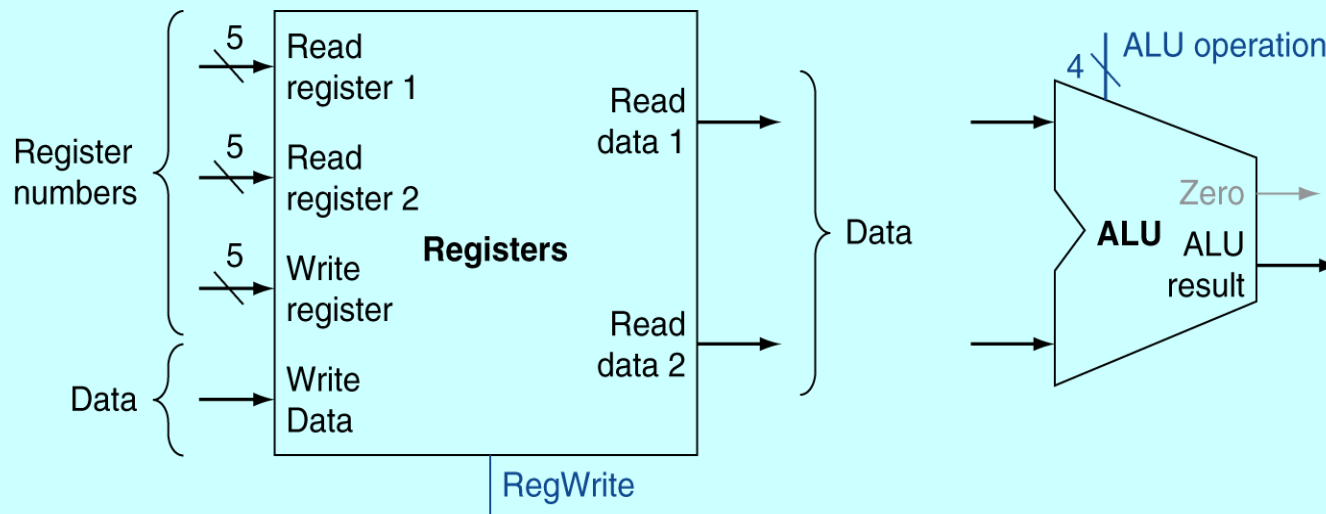
- La implementación de las distintas instrucciones tiene varias acciones en común.
- Los primeros dos pasos son iguales:
 1. Enviar el PC (contador de programa) a la memoria y sacar la siguiente instrucción (ciclo de fetch).
 2. Leer uno o dos registros.
- Lo siguiente depende de la clase de instrucción, pero es parecido sin importar el opcode exacto.

Fase FETCH



Instrucciones de Formato R

- Lee dos registros que son operandos en la instrucción
- Ejecuta operación aritmética/lógica
- Escribe resultado en el registro



a. Registers

b. ALU

Implementación

- Todas las instrucciones, excepto el salto incondicional (instrucción j), usan la **ALU** (unidad aritmético-lógica).
 - Las instrucciones de referencia a memoria lo usan para calcular direcciones.
 - Las instrucciones aritmético-lógicas para su operación.
 - Los saltos para evaluar la condición.

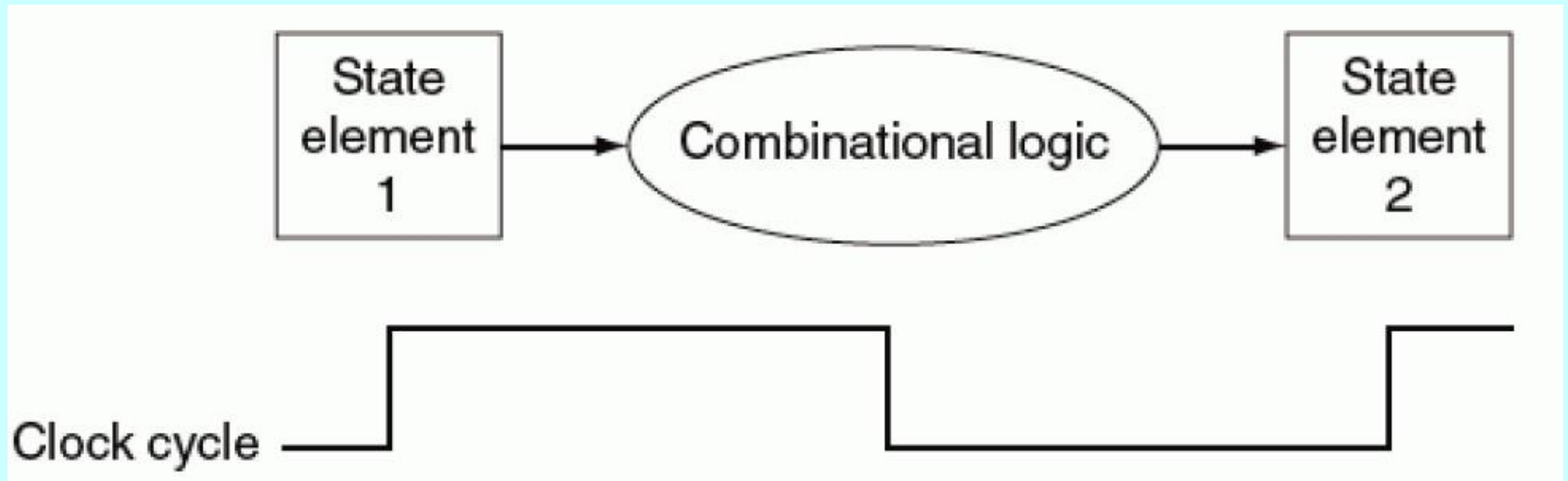
Implementación

- Después de usar la ALU:
 - Las instrucciones de referencia a memoria accesan la memoria para cargar o guardar un dato.
 - Las instrucciones aritmético-lógicas guardan el dato de la ALU en un registro.
 - Los saltos, dependiendo de la condición, cambian el contador de programa (PC) o lo incrementan en 4.

Política de reloj

- Se suponen circuitos secuenciales disparados por el flanco de reloj.
- Las partes combinatorias reciben entradas de alguna parte secuencial y escriben sus salidas en alguna otra parte secuencial.
- Los elementos secuenciales son los únicos que pueden guardar valores.

Política de reloj



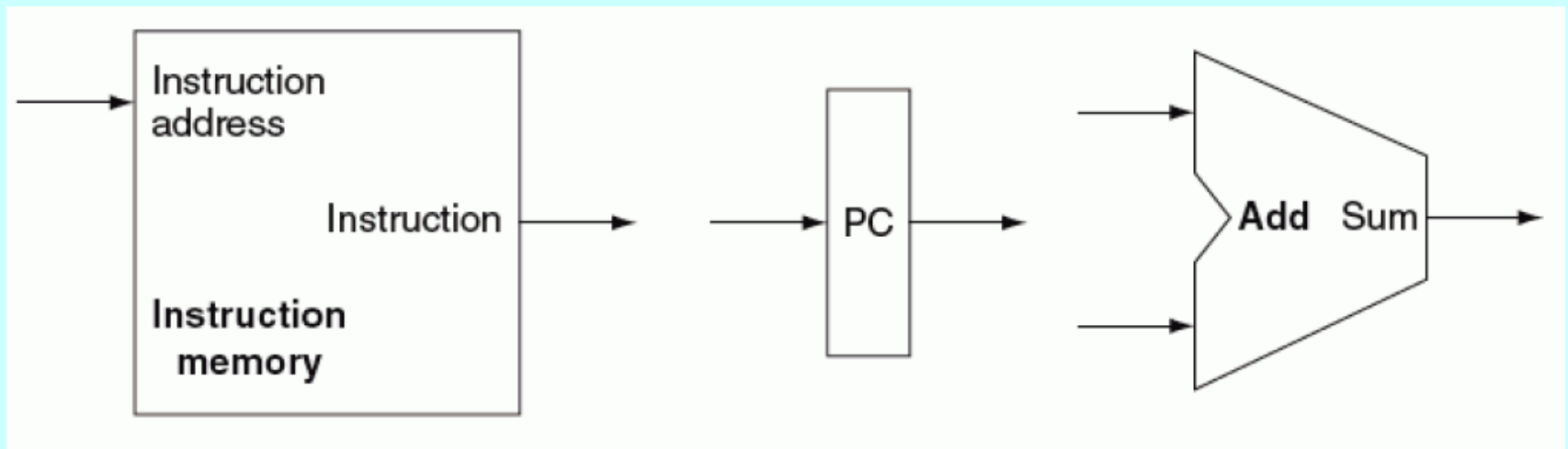
- Las señales deben propagarse de los elementos secuenciales 1 a 2 en un ciclo de reloj.
- El tiempo de propagación determina el tamaño del pulso de reloj.

Datapath

- Realiza operaciones aritméticas y lógicas.
- Elementos del datapath:
 - ALU.
 - Contador de Programa
 - Memoria de instrucciones.
 - Memoria de datos.
 - Banco de registros.
 - Sumadores.

Primeros elementos del datapath

1. Una memoria para guardar y leer instrucciones.
2. Un registro, llamado PC (contador de programa), para guardar la dirección de la instrucción actual.
3. Un sumador para incrementar el PC.

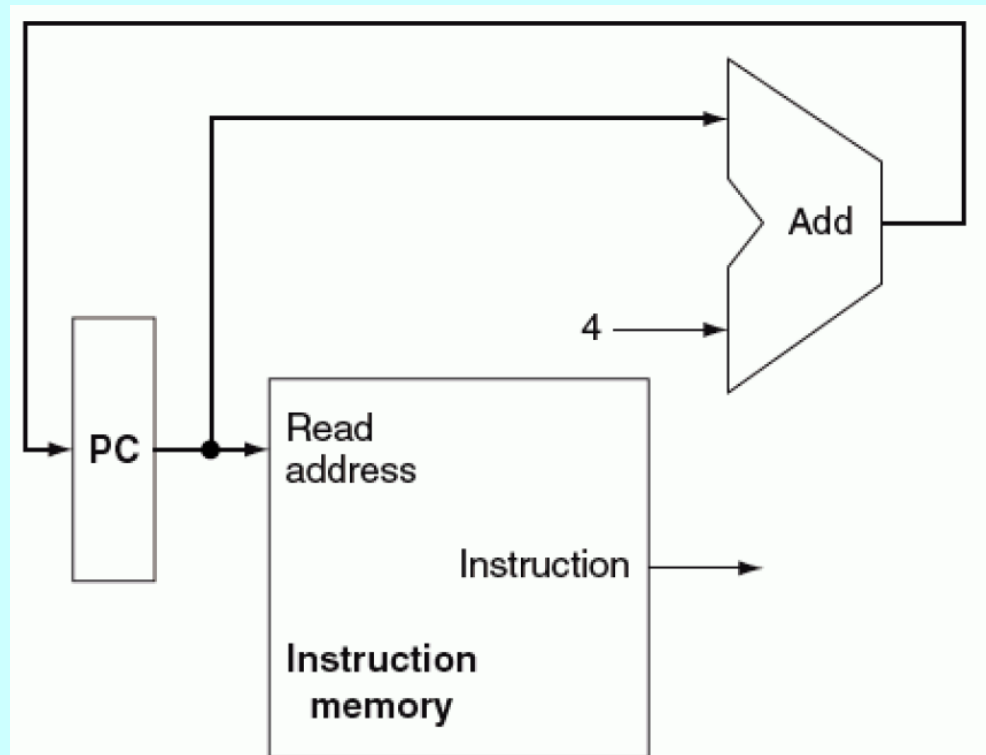


Ejecutando instrucciones

- La ejecución de una instrucción comienza con dos pasos:
 1. Obtener la instrucción de la memoria.
 2. Incrementar el PC para preparar la ejecución de la instrucción siguiente.
- Los tres elementos anteriores se combinan para formar un datapath que obtiene una instrucción e incrementa el PC.

Primera parte del datapath

- Ciclo de fetch. Lee una instrucción e incrementa el PC.



Segunda parte del datapath

- El siguiente paso es ver como se implementan:
 1. Instrucciones aritméticas y lógicas.
 2. Instrucciones de carga y almacenaje (load/store).

Instrucciones aritmético – lógicas

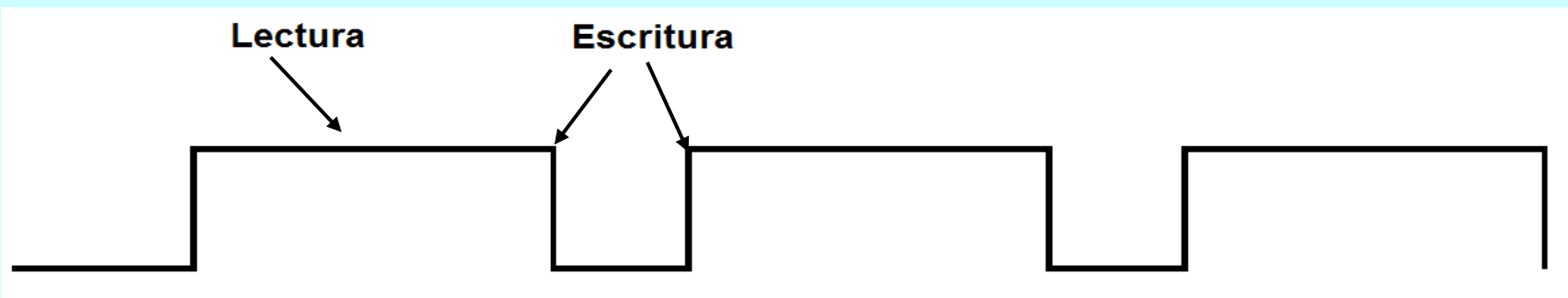
- Sus tres operandos son registros.
- También conocidas como instrucciones tipo R.
- Incluyen las instrucciones ***add***, ***and***, ***sub***, ***slt***, etc.
- Ejemplo: `add $t0, $t1, $t2 // $t0 = $t1 + $t2`
- Leen dos registros, realizan una operación aritmética o lógica y escriben el resultado en otro registro.
- Los 32 registros están guardados en el banco de registros.
- La ALU se usa para las operaciones.

Banco de registros

- Para cada instrucción, hay que leer dos palabras del banco de registros y escribir una palabra.
- Para leer un registro se indica el número de registro.
- Para escribir un registro se indica el número de registro y el dato que se va a escribir.
- Se necesitan 5 bits para especificar alguno de los 32 registros.
 $2^5 = 32$.
- Hay una señal de control que se pone a 1 para que la escritura se haga en el siguiente pulso de reloj.

Banco de registros

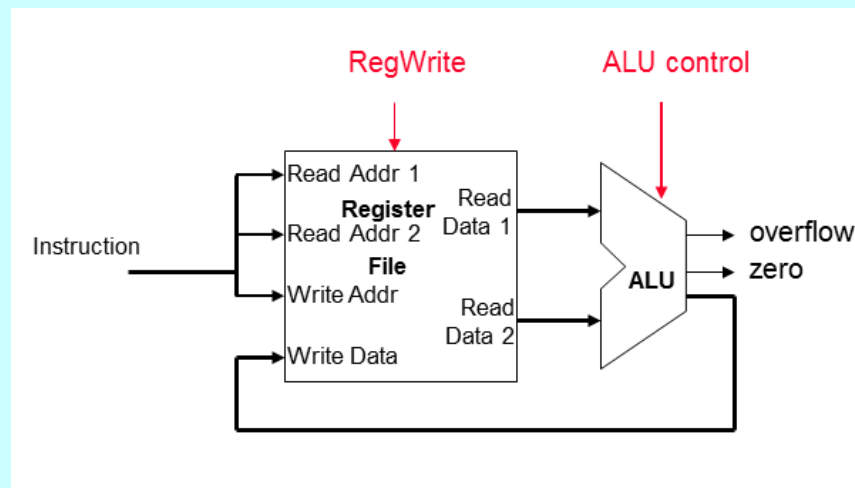
- La escritura se hace durante los flancos del reloj.
- Se puede leer y escribir el mismo registro durante el mismo ciclo de reloj.
- Se lee lo que se escribió en el ciclo anterior.
- Lo escrito está disponible en el siguiente ciclo.



Instrucciones aritmético - lógicas

En resumen, el datapath para las instrucciones aritmético-lógicas utiliza:

1. El banco de registros para leer los operandos y guardar los resultados.
2. ALU para realizar operaciones aritméticas y lógicas.



Datapath para las instrucciones de carga y almacenamiento (load/store)

- Forma:
 - `lw $r1, offset ($r2)` ; $r1 \leftarrow \text{Memoria}[r2 + \text{offset}]$
 - `sw $r1, offset ($r2)` ; $\text{Memoria}[r2 + \text{offset}] \leftarrow r1$
- Hay que sumar el offset (16 bits con signo) al registro base \$r2.
- `lw` tiene que escribir el valor en \$r1.
- `sw` tiene que leer el valor de \$r1.
- `lw` y `sw` también usan el banco de registros y la ALU.
- Además se necesita **extender el signo** al mover cantidades de 16 a 32 bits.

Extender el signo

- ¿El número 80_{16} es positivo o negativo?
- Depende...
- Si es una constante de 8 bits, 80_{16} es negativo.
$$80_{16} = 10000000_2 = -128_{10}$$
- Si es una constante de 16 bits, 80_{16} es positivo.
$$80_{16} = 0000000010000000_2 = +128_{10}$$

Extender el signo

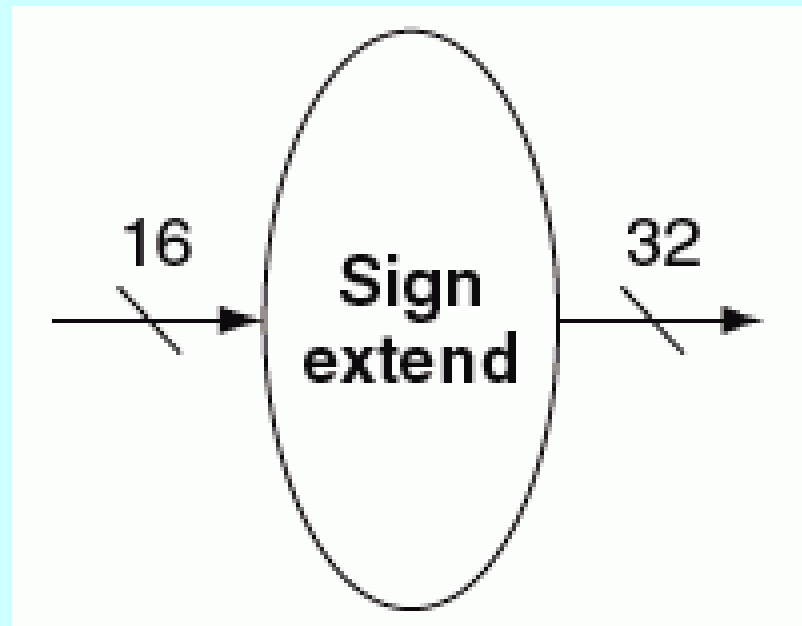
- Para copiar una constante de m bits a n bits, $n > m$, hay que tener cuidado con el signo.
- Constante de 8 bits.
 - $80_{16} = 10000000_2 = -128_{10}$
- Constante de 16 bits.
 - $0000000010000000_2 = 0080_{16} = +128_{10}$ ¡Incorrecto!
 - $1111111110000000_2 = FF80_{16} = -128_{10}$ Correcto
- El bit de signo se extiende al resto de los bits.

Extender el signo

- Otro ejemplo...
- Constante positiva de 8 bits
 - $00101011_2 = 2B_{16} = 43_{10}$
- Constante positiva de 16 bits
 - $0000000000101011_2 = 2B_{16} = 43_{10}$
- El signo positivo se extendió.

Extender el signo

- Unidad de extensión de signo.



Positivo o negativo

- En la instrucción de MIPS
 - `lw $t1, 0x8020 ($a0)`
- ¿Qué dirección de memoria hay que acceder?
- En MIPS, todas las constantes son de 16 bits.
- $8020_{16} = 10000000000100000_2 = -32736_{10}$
- El resultado de la instrucción es:
 - $t1 \leftarrow \text{Memoria } [a0 - 32736]$

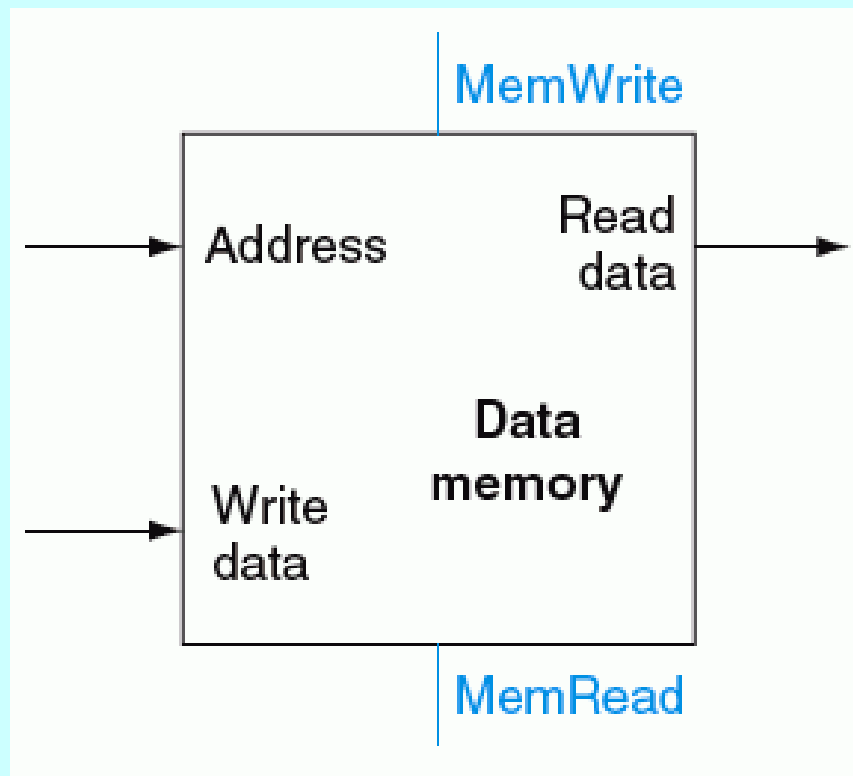
Instrucciones load/store

- Además de la unidad de extensión de signo...
- Se necesita una **memoria de datos**.
- No confundir con la memoria de instrucciones.
- La memoria de datos almacena los datos (segmento **.data** en MIPS).
- La memoria de datos es usada por:
 - La instrucción lw lee de la memoria de datos.
 - La instrucción sw escribe en la memoria de datos.

Memoria de datos

- 4 entradas:
 - Dirección del registro.
 - Dato a escribir (cuando es escritura).
 - Señal de modo de lectura.
 - Señal de modo de escritura.
- 1 salida:
 - Dato leído (cuando es lectura).

Memoria de datos



Instrucciones load/store

- En resumen, el datapath para las instrucciones load/store utiliza:
 1. El banco de registros para leer y escribir registros.
 2. ALU para sumar el registro base y el offset.
 3. La unidad de extensión de signo para mover constantes de 16 a 32 bits.
 4. Memoria de datos.

